CLASS 4

“Where Does the Truth Live?”

Understanding Client Data & Source Systems



Forward Deployed Engineering

THE NEXT CHAPTER: FROM PROBLEM TO WORKING DATA

Now We Enter the Data Phase

You scoped the problem. The next challenge is making reality usable.

CLASS 4
UNDERSTAND THE DATA
What information exists — and can we trust it?

CLASS 5
GET THE DATA
SQL · APIs · CSV · JSON · Files

CLASS 6
MAKE IT RELIABLE
Missing values · duplicates · validation

CLASS 7
MODEL THE BUSINESS
Turn raw records into entities and relationships

CLASS 8
BUILD THE PIPELINE
Source → transformation → usable system

Problem → **Information** → Retrieval → Quality → Model → Pipeline

> *An FDE does not start by building the pipeline. First, they decide what truth the pipeline must carry.*

CLASS 4 | CLASS 5 | CLASS 6 | CLASS 7 | CLASS 8

ONE PROBLEM — FIVE DATA QUESTIONS

The same question, five classes deep.

CUSTOMER

“Where is my refund?”

Today we solve only the first question:
Where does the truth live?

CLASS 4 — UNDERSTAND
Which systems contain customer, order, refund and policy information?

CLASS 5 — RETRIEVE
How do I actually fetch those records using SQL, APIs and files?

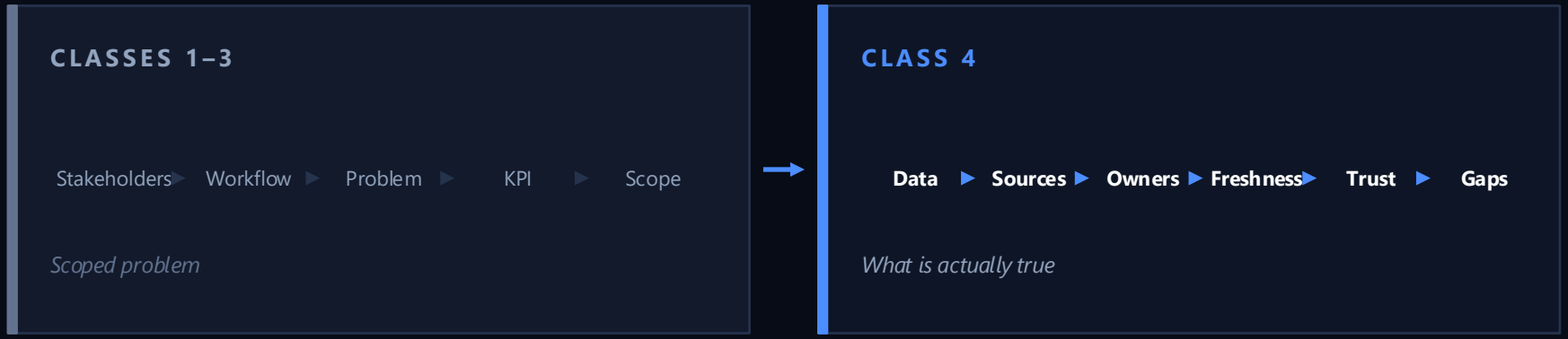
CLASS 6 — TRUST
What if IDs are missing, data conflicts, or timestamps are stale?

CLASS 7 — MODEL
How are Customer → Order → Refund → Ticket related?

CLASS 8 — PIPELINE
How do we keep that information fresh and dependable in production?

FROM SCOPE TO DATA

We know WHAT we want to solve.
Now we need to know WHAT IS TRUE.



> *A scoped problem tells you what information you need. It does not tell you where that information lives.*

TODAY'S SCOPE Data → Sources → Owners → Freshness → Trust → Gaps

No SQL yet. First decide what is worth querying.

ShopKart has a refund-support problem.

CUSTOMER

"Where is my refund?"

OUR SCOPED OBJECTIVE

Help agents reliably understand the current refund state.

BUSINESS PROBLEM

Long resolution times

Repeated customer contacts

Agents switching between systems

So — what information would you need before building anything?

THE CLIENT DROPS THE DATA FOLDER

Where would you start?

CSV
tickets.csv
Support platform export

CSV
orders.csv
Daily OMS export

JSON
refunds.json
Refund service export

PDF
support_policy.pdf
Refund & returns policy

XLSX
agent_notes.xlsx
Free-text agent notes

SYSTEM
CRM
Customer profiles

DATABASE
Order DB
Live order data

API
Refund API
Live refund lifecycle

Having data ≠ understanding data.

POLL 1 · DATA DETECTIVE

Which sources matter for the refund problem?

A customer asks:

“Where is my refund?”

Which sources could contain information relevant to resolving this request?

MULTI-SELECT

Select every source you believe may matter.

- **A** tickets.csv
- **B** orders.csv / Order DB
- **C** refunds.json / Refund API
- **D** support_policy.pdf
- **E** agent_notes.xlsx
- **F** CRM
- **G** All available company datasets should be ingested

📊 **POLL TIME — Open the class poll and submit your answer before we discuss.**

BREAK THE PROBLEM INTO INFORMATION NEEDS

Don’t start with the data sources.

Start with the questions the business problem requires you to answer.

WHERE IS MY REFUND?

1	**WHO IS ASKING?**	Customer identity
2	**WHICH PURCHASE?**	Order identity / context
3	**WAS A REFUND CREATED?**	Refund existence
4	**WHAT STATE IS IT IN?**	Requested · Approved · Processing · Completed · Failed
5	**WHEN DID IT CHANGE?**	Timestamps / freshness
6	**WHAT SHOULD HAPPEN NEXT?**	Policy / eligibility / escalation

INFORMATION NEED
“What state is the refund in?”

REQUIRED FIELD
refund_status

DATA SOURCE
Refund Service

SOURCE OWNER
Finance Team

First decide WHAT you need to know.
Then ask WHERE that truth lives.

Problem → Questions → Information → Fields → Sources

REVEAL

Different questions require different sources.

CUSTOMER QUERY

Ticket context	→ **Support Platform**
Customer identity	→ **CRM**
Order context	→ **Order DB**
Refund state	→ **Refund Service**
Policy / eligibility	→ **Policy Document**
Operational context	→ **Agent Notes**

Relevant does not mean authoritative.

STRUCTURED
SQL / CSV

SEMI-STRUCTURED
JSON / APIs

UNSTRUCTURED
PDF / Notes

Start from the information the problem requires — not from every dataset the client owns.

POLL 2 · BUILD THE DATA-SOURCE MAP

Who owns the truth?

Match each information need to the source you would investigate FIRST. **Use the chain: Information Need → Field → Source**

Q1

Customer identity

- **A** CRM
- **B** Agent notes
- **C** Refund API
- **D** Policy PDF

Q2

Live order status

- **A** Daily orders.csv export
- **B** Order Management DB
- **C** CRM
- **D** Support ticket

Q3

Refund lifecycle state

- **A** CRM
- **B** Refund Service API
- **C** Agent notes
- **D** orders.csv

Q4

Refund eligibility / policy

- **A** Refund API
- **B** Policy document
- **C** CRM
- **D** tickets.csv

📊 **POLL TIME — Map the information need to its likely owner.**

Map information → source → owner.



Capture for each system: Owner · Format · Freshness · Access · Reliability

Every critical field should have an owner and a source.

POLL 3 · SOURCE-OF-TRUTH CHALLENGE

11:07 AM — An angry customer is waiting.

> “My refund was completed yesterday. Why don’t I have my money?”

CRM

APPROVED

Today, 08:00

refunds.json

PROCESSING

Yesterday, 18:00

Refund Service API

COMPLETED

Today, 10:42

Agent note

“Finance said completed”

Today, 10:50

Q1 — Which source would you tentatively treat as the system of record for refund state?

A CRM

B refunds.json

C Refund Service API

D Agent Notes

E Whichever was updated most recently

Q2 (multi-select) — What should influence your source-of-truth decision?

A Which system owns the refund lifecycle

B Freshness / update frequency

C Whether another system is only a copy

D Who manually edits the field

E Whether the data is convenient to query

F Business definition of each status

📊 **POLL TIME — Decide before seeing the reasoning.**

REVEAL · SOURCE OF TRUTH

The newest value is not automatically the truth.

CRM	COPY / SYNCED VIEW
CSV / JSON EXPORT	SNAPSHOT
AGENT NOTE	HUMAN CONTEXT
REFUND SERVICE	OWNS REFUND LIFECYCLE

→ **REFUND SERVICE API is our tentative System of Record**

OWNERSHIP
Which system creates and changes this business object?

FRESHNESS
How quickly does the value change?

REPLICATION
Is this source only copying another source?

SEMANTICS
Does “approved” mean the same as “completed”?

A system of record is a business + system decision, not just a technical one.

If the CRM updated at 10:55, would it suddenly become the truth?

Freshness alone is not authority.

POLL 4 · REQUIRED FIELDS

What is the MINIMUM data our MVP needs?

MVP SCOPE

Answer refund-status questions for authenticated customers.

MULTI-SELECT

Which fields are truly required for the first version?

Select only what you believe is necessary.
More data is NOT automatically better.

- **A** customer_id
- **B** order_id
- **C** refund_id
- **D** refund_status
- **E** refund_requested_at
- **F** refund_updated_at
- **G** customer_date_of_birth
- **H** complete customer purchase history
- **I** product recommendation history
- **J** ticket_id
- **K** current support-agent performance score

📊 **POLL TIME — Choose the minimum viable data.**

REVEAL · REQUIRED FIELDS

Start from the decision. Work backwards to the fields.

BUSINESS QUESTION

“Where is my refund?”

↓

Identify customer / order

↓

Find refund

↓

Read current state

↓

Understand when it changed

↓

Respond / escalate

LIKELY MINIMUM SET

- customer_id
- order_id
- refund_id
- refund_status
- refund_requested_at
- refund_updated_at
- ticket_id / conversation context

Additional fields may become necessary after validation.

Don’t ingest every field because access exists.

BUSINESS QUESTION → DECISION WE NEED TO MAKE → MINIMUM INFORMATION → FIELDS

THE TWIST

Then you discover this...

Tickets and the Order Database DO NOT share a reliable customer_id.

TICKET
customer_email
priya@gmail.com



ORDER DB
customer_id
C-84920

no reliable join key

- Customers use different email addresses
- Shared family accounts exist
- Agents paste order IDs into free-text notes
- Some tickets contain no order number

Missing data can invalidate a good product idea — some data problems are workflow problems.

ENGINEER AROUND IT
Entity resolution / mapping

CHANGE THE WORKFLOW
Capture order_id at ticket creation

1. Is this a data-quality problem?
2. Is this a workflow problem?
3. Does this change our MVP scope?

Find another reliable identifier

YES → validate the mapping

NO → change workflow / collect ID

Revisit MVP scope

What would YOU do before writing a join?

WHAT AN FDE SHOULD LEAVE WITH

Before touching the pipeline...

- What problem?
- What information?
- Where does it live?
- Who owns it?
- How fresh is it?
- Can I trust it?
- What is missing?

THREE EXPECTED OUTPUTS

DATA-SOURCE MAP

Information need → source → owner

REQUIRED-FIELDS LIST

The minimum the MVP truly needs

SOURCE-OF-TRUTH DECISIONS

With the reasoning written down

APPLIED CHALLENGE · 20–25 MIN

“Why Is My Food Late?”

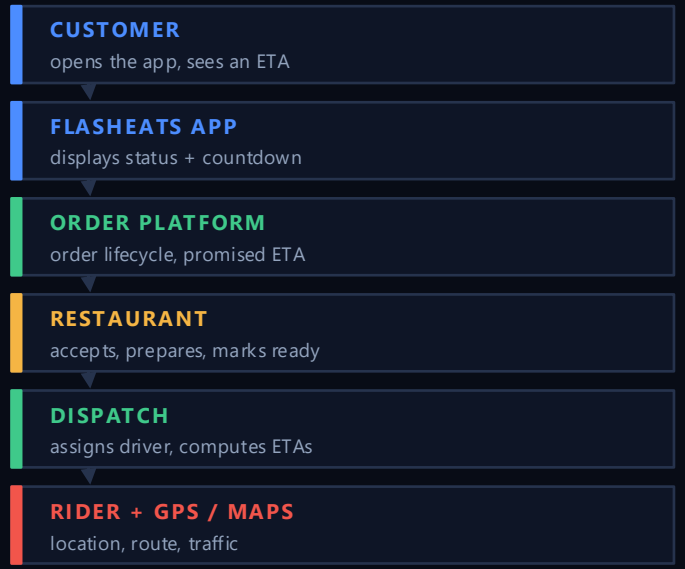
You are the FDE team for FlashEats.

CLIENT REQUEST

“Our customers are angry about late deliveries. Build an AI system that predicts delays and automatically fixes the problem.”

Before building anything —
what do we actually need to understand?

No architecture. No model. Only the information behind the promise.



DO NOT DESIGN THE AI YET.

One order. Seven different versions of reality.

WHY IS MY FOOD LATE? every answer below lives in a different system, with a different owner.

CUSTOMER APP

displayed ETA
order status

RESTAURANT API

accepted_at · prep_started_at
ready_at · restaurant_status

status often updated manually

DRIVER LOCATION API

GPS coordinates
timestamp · driver status

GPS can be stale by 2–5 min

DISPATCH SERVICE

assigned_driver · reassignment
pickup ETA · delivery ETA

MAPS / TRAFFIC API

route duration
congestion

ORDER PLATFORM / orders.csv

order lifecycle · promised ETA
delivered_at · final status

30-minute export lag

SUPPORT TICKETS

free-text customer complaints

unstructured + subjective

Available ≠ reliable ≠ authoritative.

DECOMPOSE THE PROBLEM

Don’t start with the systems.

Start with the questions required to explain a delay.

WHY IS THE ORDER LATE?

- Was food preparation late?
- Was driver assignment late?
- Did the driver reach the restaurant late?
- Was pickup delayed?
- Is traffic causing the delay?
- Is the displayed ETA itself wrong / stale?

Questions first. Fields and systems only after.



YOUR TURN — Build this map before looking at individual fields.

LIVE INCIDENT · ORDER #FE-48291

7:31 PM — The customer is getting angry.

"Your app has said 10 minutes away for the last 20 minutes. Where is my food?"

ORDER #FE-48291

CUSTOMER APP
ETA: 7:42 PM
Status: Driver picking up

RESTAURANT API
Status: PREPARING
Last update: 7:21 PM

RESTAURANT EMPLOYEE
"Food has been ready for 12 minutes."

DRIVER API
1.8 km from restaurant
Last GPS: 7:29 PM

DISPATCH SERVICE
Pickup ETA: 7:34 PM
Delivery ETA: 7:51 PM

MAPS API
Traffic delay: +11 min

SUPPORT TICKET
"App keeps changing the ETA."

WHAT IS ACTUALLY CAUSING THE DELAY?

Can you answer confidently yet?

📊 **POLL TIME —** Open the class poll and submit your answer before we discuss.

FDE CHALLENGE · WORK INDIVIDUALLY OR IN PAIRS

Make the call.

Produce six deliverables for FlashEats — reasoning on paper, no tooling.

1 INFORMATION NEEDS

What must we know to explain the delay?

2 SOURCE MAP

Which source should provide each piece of information?

3 SYSTEMS OF RECORD

Delivery ETA · driver assignment · restaurant readiness · delivery completion

4 DATA RISKS

Name at least 4: stale · manual · conflicting · incomplete · ambiguous

5 MISSING INFORMATION

What prevents you from confidently explaining the delay?

6 REVISED PROBLEM

Is this really an "AI prediction problem"?

20 MINUTES

Clarity of reasoning > number of fields.

THE TWIST

Then engineering discovers this…

The platform does NOT reliably store when the driver actually arrived at the restaurant.

driver_assigned_at	recorded
GPS samples	approximate
??? DRIVER_ARRIVED_AT_RESTAURANT ???	missing
pickup_confirmed_at	recorded
delivered_at	recorded

Can we reliably tell whether the restaurant or the driver caused the delay?

- **A** Infer arrival using GPS + geofence
- **B** Add a driver "Arrived" event
- **C** Automatically instrument restaurant arrival
- **D** Change MVP scope until the event exists

Before building clever matching logic — could we change the workflow?

Some data problems are actually instrumentation or workflow problems.

DEBRIEF

What changed?

CLIENT ASKED FOR

"Build an AI system that predicts delays."

WHAT WE MAY HAVE DISCOVERED

"We cannot reliably attribute why an order is late because critical operational events are missing or inconsistent across systems."

Same method. New client. Harder system.

1. Which source did you trust for ETA — and why?
2. Which source worried you most?
3. What information was missing?
4. Would you build the AI delay predictor today?

What should we fix FIRST?

FDE skill = changing your understanding when the data changes.

INSTRUCTOR REVEAL — DO NOT SHOW BEFORE DISCUSSION

A strong FDE interpretation

DELIVERY ETA — **Dispatch Service**

It creates the platform ETA; the app displays it. Maps is an input, not the business ETA.

DRIVER ASSIGNMENT — **Dispatch Service**

Assignment and reassignment originate here.

DRIVER LOCATION — **Driver Location API**

Owner is clear — freshness and GPS reliability are not.

RESTAURANT READY STATE — **Restaurant system**

Officially owns it, but updates are manual, so the data may be unreliable.

DELIVERY COMPLETION — **Order / delivery platform**

Depends on where the completion event is actually raised.

MOST IMPORTANT GAP: no reliable driver-arrived-at-restaurant event.

POSSIBLE REVISED PROBLEM

"FlashEats cannot reliably attribute delivery delays because key operational events — particularly restaurant arrival and readiness — are missing or inconsistently recorded."

BEFORE BUILDING AN AI DELAY PREDICTOR

1. Improve operational event instrumentation
2. Define canonical ownership for major delivery states
3. Validate timestamp freshness and semantics
4. Establish reliable delay labels
5. Only then evaluate prediction

AI cannot reliably predict a label the system itself cannot define.

CONSOLIDATION

What did we actually learn today?

Client problem

↓

What do we need to know?

↓

Where does that information live?

↓

Who owns it?

↓

Which source can we trust?

↓

What information is missing?

↓

Does the gap change our scope?

1 **START FROM THE PROBLEM**
Not from the dataset.

2 **MAP INFORMATION TO SOURCES**
Different questions have different owners.

3 **RELEVANT ≠ AUTHORITATIVE**
Not every useful source is the system of record.

4 **MINIMUM DATA > ALL DATA**
Only collect what the decision needs.

5 **MISSING DATA IS A PRODUCT SIGNAL**
Sometimes change the workflow or scope.

Querying the wrong source perfectly is still wrong.

NEXT CLASS
Now we fetch the data — SQL, APIs & Files.

PROBLEM | MESSY DATA | WHAT MATTERS | CONFLICTS | TRUST | MISSING | REVISE